MODULE 2 : Création d'une API REST avec Node.js

Objectifs :

- Comprendre l'écosystème web backend de Node.js.
- Maîtriser le pattern "Middleware" (fondation de l'ingénierie web moderne).
- Implémenter le concept "Zero Trust" via la validation stricte des inputs (Zod/Ajv).
- Créer une architecture BFF (Backend For Frontend).

2.1. L'Écosystème Node.js pour le Web : Lequel choisir ?

Node.js possède un module natif `http`, mais il est trop bas niveau (complexe) pour créer une API rapidement. On utilise donc des frameworks.

- **Express.js (Le Standard Historique) :**
  - *Avantages :* C'est le roi incontesté. la plupart des entreprises l'utilisent. L'écosystème de plugins est gigantesque.
  - *Inconvénients :* Son cœur est ancien. Il gère nativement mal les erreurs asynchrones (bien que la v5 corrige cela) et ses performances pures sont moyennes par rapport aux standards actuels.
- **Fastify (Le Challenger Haute Performance & Sécurité) :**
  - *Avantages :* Construit pour la vitesse. Mais surtout : **il intègre nativement la validation par JSON Schema**. Il compile les routes à l'avance et possède des limites strictes par défaut (ex: taille maximale de payload) qui protègent nativement contre certains dénis de service (DoS).
- **Koa.js (L'Élégance Minimaliste) :**
  - Créé par l'équipe d'Express pour corriger ses défauts avec les Promises. Très léger, mais demande d'installer beaucoup de modules tiers pour faire l'équivalent d'Express.

2.2. Architecture d'une API : Routage et Middlewares

A. Le Routage (Routing)

C'est l'aiguillage de l'application. Comment le serveur traduit une requête texte (ex: `GET /users/123?active=true`) en une fonction JavaScript ?

- **La Route / Le Chemin (Path) :** L'URL de base (ex: `/users`).
- **Le Paramètre d'URL (Path Parameter) :** `/users/:id`. Identifie une ressource précise. L'ingénieur doit toujours vérifier si `:id` est bien un format attendu (ex: un UUID ou un entier), sinon c'est une porte ouverte aux injections.
- **La Chaîne de requête (Query String) :** `?active=true&sort=asc`. Utilisée pour filtrer, trier ou paginer. *Ne jamais utiliser pour passer des données sensibles.*

B. Le Pattern Middleware (L'Oignon de la sécurité)

C'est le concept architectural le plus important de Node.js.

- **Le concept :** Un middleware est une fonction qui a accès à la requête (`req`), à la réponse (`res`), et à la fonction suivante (`next`).
- **L'analogie de l'Aéroport :**
  - *Requête entrante :* Un passager arrive à l'aéroport.
  - *Middleware 1 (Rate Limit) :* L'aéroport n'est-il pas trop plein ? (Si oui : `429 Too Many Requests`).
  - *Middleware 2 (Authentification) :* Le passager a-t-il un billet valide ? (Si non : `401 Unauthorized`).
  - *Middleware 3 (Validation Zod) :* Le bagage (Payload) respecte-t-il la taille et le contenu autorisé ? (Si non : `400 Bad Request`).
  - *Contrôleur Final :* Le passager embarque dans l'avion (La base de données est modifiée, on renvoie `200 OK`).
- **Règle d'or :** En sécurité, on veut **Fail Fast** (échouer le plus tôt possible). Inutile d'ouvrir une connexion à la base de données si le payload contient des caractères illégaux. Le middleware bloque la requête avant même que le contrôleur métier ne soit appelé.

2.3. Validation des données : La règle du "Zero Trust"

- **Pourquoi valider ?** (CWE-20 : Improper Input Validation). Si on attend un âge et que l'utilisateur envoie `{"age": {"$gt": 0}}`, une base MongoDB non sécurisée exécutera cette commande (NoSQL Injection) et renverra tous les utilisateurs !
- **L'approche déclarative (Schémas) :** Au lieu de faire des dizaines de `if (req.body.age > 18)`, l'ingénieur moderne déclare un **Schéma** (un contrat mathématique).
- **Présentation de Zod :**
  - Librairie moderne, "TypeScript-first".
  - Permet de définir des contraintes strictes : `z.string().email().min(5).max(100)`.
  - Si la donnée ne correspond pas, Zod génère un tableau d'erreurs extrêmement détaillé que l'on peut renvoyer directement au client (Erreur HTTP 400).

PARTIE 2 : PRATIQUE

Atelier 3 : Initialisation du serveur et CRUD

Objectif : Mettre en place la fondation Express et comprendre les verbes HTTP côté serveur.

1. Initialisation du projet :

```
npm init -y
npm install express
npm install -D nodemon # Pour redémarrer le serveur à chaque sauvegarde
```

Ajouter `"dev": "nodemon index.js"` dans les scripts du `package.json`.

2. Le code de base (`index.js`) :

```
const express = require("express");
const app = express();
const PORT = 3000;

// Middleware global natif pour dire à Express de parser le JSON entrant
app.use(express.json());

// Notre base de données fictive
let products = [
  { id: 1, name: "PC Portable", price: 1200 },
  { id: 2, name: "Souris", price: 25 },
];

// 1. READ (Tous les produits)
app.get("/api/products", (req, res) => {
  res.status(200).json(products);
});

// 2. READ (Un seul produit via Paramètre d'URL)
app.get("/api/products/:id", (req, res) => {
  // 🚨 Attention : req.params.id est toujours une String !
  const productId = parseInt(req.params.id, 10);
  const product = products.find((p) => p.id === productId);

  if (!product) {
    return res.status(404).json({ error: "Produit introuvable" });
  }
  res.status(200).json(product);
});

// 3. CREATE
app.post("/api/products", (req, res) => {
  // Pour l'instant, on fait une confiance aveugle au body (TRÈS MAUVAIS !)
  const newProduct = {
    id: products.length + 1,
    name: req.body.name,
    price: req.body.price,
  };
  products.push(newProduct);

  // Bonne pratique : 201 Created + on renvoie l'objet créé
  res.status(201).json(newProduct);
});

app.listen(PORT, () => {
  console.log(`🚀 Serveur démarré sur http://localhost:${PORT}`);
});
```

Atelier 4 : Implémentation de la validation avec Zod

Objectif : Transformer le code vulnérable précédent en forteresse en implémentant le pattern Middleware + Zod.

1. Installation : `npm install zod`

2. Création du Schéma de Validation : On va obliger le client à envoyer exactement ce qu'on attend, ni plus, ni moins.

```
const { z } = require("zod");

// Définition du contrat mathématique
const productSchema = z
  .object({
    name: z.string().min(3, "Le nom doit faire au moins 3
caractères").trim(),
    price: z.number().positive("Le prix doit être positif"),
    category: z.enum(["informatique", "bureau"]).optional(),
  })
  .strict(); // .strict() interdit l'envoi de champs non déclarés (ex:
isAdmin)
```

3. Le Middleware de Validation Magique :

```
// Middleware qui prend un schéma Zod en paramètre
const validateData = (schema) => {
  return (req, res, next) => {
    try {
      // parse() lève une exception si les données sont invalides
      // Il nettoie aussi les données (trim) et les remplace dans req.body
      req.body = schema.parse(req.body);
      next(); // Tout est OK, on passe au contrôleur suivant
    } catch (error) {
      // ZodError : on formate joliment l'erreur pour le client
      return res.status(400).json({
        error: "Données invalides",
        details: error.errors.map((e) => `${e.path.join(".")} :
${e.message}`),
      });
    }
  };
};
```

4. Sécurisation de la route POST : On insère notre middleware entre l'URL et le contrôleur.

```
// La route est maintenant protégée !
app.post("/api/products", validateData(productSchema), (req, res) => {
  const newProduct = {
    id: Math.floor(Math.random() * 1000), // Faux ID
```

```
    ...req.body, // On peut utiliser le destructuring en toute sécurité
grâce à Zod
  };
  products.push(newProduct);
  res.status(201).json(newProduct);
});
```